Лабораторная работа №1.

Динамическое программирование на подмножествах. Сегментация текста с весовой функцией.

Цель работы

Освоить метод динамического программирования на подмножествах для решения задачи сегментации последовательности. Изучить способы модификации классического алгоритма «Word Break» для работы с весами слов, с ограничениями на структуру словаря и с нечеткими совпадениями. Научиться восстанавливать оптимальное разбиение по нескольким критериям.

Теоретическая часть

1. Постановка задачи

Пусть дана строка **S** (текст без пробелов и знаков препинания) длины **n**. Дан словарь **D**, содержащий набор слов. Каждому слову **w ∈ D** сопоставлен вес (частота, стоимость, вероятность) — целое или вещественное число **weight(w)**. Требуется разбить строку **S** на последовательность слов **(w1, w2, ..., wk)** таких, что:

1. Конкатенация **w1 + w2 + ... + wk** в точности равна **S**.
2. Каждое **wi ∈ D**.
3. Сумма весов использованных слов максимальна (или, в зависимости от варианта, минимальна или удовлетворяет иным критериям).

2. Динамическое программирование на подмножествах

В классическом понимании «ДП на подмножествах» — это перебор всех подмножеств элементов с помощью битовых масок (например, задача коммивояжера). В данном контексте мы имеем дело с линейной структурой (строка), но суть та же: состояние определяется множеством уже обработанных символов, которое задается индексом. Каждый следующий шаг — это выбор слова, «покрывающего» непрерывный отрезок строки.

Формально, состояние **dp[i]** — это оптимальное значение (максимальная сумма весов) для префикса строки **S[0..i-1]** (первые **i** символов). Это частный случай ДП на подмножествах, где множество представлено в виде линейного порядка, но переходы осуществляются скачками (словами разной длины).

3. Базовый алгоритм (за **O(n^2 * L),** где **L** — длина слова)

Для решения задачи используется одномерный массив **dp** размером **n+1**. Индексация: **dp[0] = 0** (пустая строка — вес 0).
Для каждого **i** от 1 до **n** (рассматриваем префикс длины **i**):
Для каждого **j** от 0 до **i-1** (место предполагаемого предыдущего разрыва):

- Извлекаем подстроку **candidate = S[j:i].**

- Если **candidate ∈ D**, то обновляем **dp[i]**:
- **dp[i] = max(dp[i], dp[j] + weight(candidate)).**

Сложность: **O(n^2 * L)** на поиск подстроки в словаре, что для длинных строк (n ~ 10^5) неприемлемо.

4. Оптимизации

1. **Хэш-таблица:** Храним словарь в хэше. Проверка вхождения за **O(1),** но копирование подстроки все еще **O(L).** Можно использовать хэширование строк (полиномиальный хэш), чтобы получать хэш любой подстроки за **O(1).**

2. **Бор (Trie / Префиксное дерево):** Вместо перебора всех **j**, мы идем от **i** вперед и спускаемся по бору. Как только находим конец слова в боре на позиции **k**, делаем переход **dp[k] = max(dp[k], dp[i-1] + weight).** Это снижает сложность до **O(n * max_len_dict)**, где **max_len_dict** — максимальная длина слова в словаре.

3. **Суффиксный массив:** Для самых сложных случаев, когда словарь очень большой, используются алгоритмы множественного поиска подстрок.

5. Восстановление ответа

Для восстановления последовательности слов используется массив предков **prev[i],** где хранится:

- Индекс начала последнего слова **(j),** которое привело в состояние **i.**
- Или само слово, чтобы потом его вывести.

Требования к реализации (общие для всех вариантов)

1. **Ввод данных:**
- Строка **S** (может содержать только строчные латинские буквы для простоты).
- Словарь: сначала число **M**, затем **M строк** вида **слово вес**.
- Для варианта 2 дополнительно матрица **MxM**.
2. **Вывод:**
- Если разбиение невозможно: -1.
- Если возможно: первая строка — максимальная сумма (или минимальный штраф), вторая строка — слова разбиения, разделенные пробелом (в случае омонимии можно выводить слово с пометкой значения, например, **bow(лук)).**
3. **Ограничения:**
- Длина строки **S** до 10^5.
- Количество слов в словаре до 10^4.
- Время работы должно быть близко к **O(n * L_max)** с использованием бора или хэшей.

Индивидуальные варианты
Каждый вариант представляет собой модификацию целевой функции или структуры данных.

Вариант 1. Классический (Максимум суммы)
Цель: Максимизировать сумму весов слов.
Вход: Строка **S**, словарь (слово -> вес).
Выход: Максимальная сумма и разбиение.
Особенность: Если разбиение невозможно, вернуть -1.

Вариант 2. Минимум штрафа за разрыв (Сегментация с контекстом)
Цель: Минимизировать суммарный штраф.
Данные: Словарь (слово -> базовая стоимость). Дополнительно задана матрица **penalty[a][b]** — штраф за то, что за словом **a** следует слово **b**. Штрафы могут быть отрицательными (поощрение).
Выход: Минимальная сумма штрафов + последовательность.

Вариант 3. Ограниченный словарь (Длина слова ≤ L)
Цель: Максимизация суммы.
Условие: Максимальная длина слова в словаре равна **L** (**L** вводится пользователем или вычисляется автоматически).
Задача: Реализовать оптимизированный алгоритм, который не перебирает все **j** от **i-1** до 0, а только **j** от **i-L** до **i-1**. Это снижает сложность до **O(n * L).**

Вариант 4. Омонимия и множественные пути (Нечеткий словарь)
Цель: Найти разбиение с максимальной суммой, учитывая, что одно и то же написание может соответствовать разным словам (с разными весами).
Пример: Слово "ключ" может быть (родник, 10) и (инструмент, 8).
Задача: При совпадении подстроки, нужно рассмотреть все возможные значения слова (веса) и выбрать лучшее для данного контекста (но контекст здесь — только предыдущие слова, то есть сумма нарастающим итогом).

Вариант 5. Префиксная независимость (Запрет на вложения)
Цель: Максимизация суммы.
Условие: Словарь устроен так, что никакое слово не является префиксом другого слова (например, набор синонимов или специально подобранный технический словарь).
Задача: Использовать структуру "префиксное дерево (бор)" для быстрого поиска. Если ни одно слово не является префиксом другого, то на любой позиции в строке может закончиться не более одного слова из словаря, что упрощает обработку.

Тесты для проверки
Вариант 1: Максимум суммы
Словарь:
cat: 5
cats: 8
and: 3
sand: 6
dog: 4
catsanddog: 20

Входная строка	Ожидаемый результат	Ожидаемое разбиение
catcat	**10**	**[«cat», «cat»]**
xxx	**-1**	**невозможно**

Вариант 2: Минимум штрафа за разрыв
Словарь: a:1, b:1, c:1
Матрица штрафов (от a к b):

- a->a: 0
- a->b: 1
- a->c: 2
- b->a: 2
- b->b: 0
- b->c: 1
- c->a: 1
- c->b: 2
- c->c: 0

Входная строка	Ожидаемый мин.штраф	Ожидаемое разбиение
ab	**1**	**[«a», «b»]**
acb	**4**	**[«a», «c», «b»]**

Вариант 3: Ограниченный словарь (L=3)
Словарь (макс. длина 3):

- ab:2
- abc:5
- bc:3
- ca:4
- a:1
- b:1
- c:1

Входная строка	Ожидаемая сумма	Пояснение
abc	**5**	**[«a,b,c»]**

ababc	**7**	**[«ab», «abc»]**

Вариант 4: Омонимия (Несколько значений)
Словарь:

- "bow": (лук) вес 5
- "bow": (наклон) вес 3
- "tie": (галстук) вес 4
- "tie": (ничья) вес 6
- "break": (ломать) вес 7
- "break": (перерыв) вес 8

Входная строка	Ожидаемая сумма	Ожидаемое разбиение
bow	**5**	**«лук» (выбираем макс вес)**
bowtie	**11**	**«лук» + «ничья»**

Вариант 5: Префиксная независимость (бор)
Словарь:

- cat
- dog
- house
- tree
- car
- bike

Входная строка	Ожидаемый результат	Примечание
cat	**[«cat»]**	**OK**
ca	**-1**	**невозможно**